Тема 3. Итерационное программирование на LISP

Виды циклов

Бывают следующие виды циклов:

- специализированные стандартные циклы (dolist - обойти список, dotimes - от нуля до N-1, ...)
- общий стандартный цикл (do, do*)
- простая форма loop (безконечный цикл с выходом по return)

Функция DOLIST

Функция организует "пробег" переменной по заданному списку. В современных языках программирования такую организацию цикла принято называть циклом "for each". Синтаксис вызова функции DOLIST :

```
(dolist (итератор (список) результат)
                (форма1)
                (форма2)
                …
                (формаn)
)
```

Функция последовательно присваивает переменной-итератору значения элементов списка, после чего выполняется тело цикла (форма1) - (форман). После завершения функция DOLIST возвращает результат, заданный формой результат. Вот пример, поясняющий все сказанное:

```
(dolist (ite '(1 2 3 4 5 6 7 8 9 10) 'Ok)
   (printline (fact ite))
)

1
2
6
24
120
720
5040
40320
362880
3628800

==> ok
```

Естественно, вызовы функций DOLIST могут быть вложенными. При этом допустимо использовать одну и ту же переменную-итератор (ситуация, немыслимая в традиционных языках программирования!). Правда, в этом случае во внутреннем цикле не будет доступно "внешнее" значение итератора:

```
(dolist (ite '(1 2 3) 'ok)
  (dolist (ite '(4 5 6) 'ok)
    (printline ite)))

4
5
6
4
5
6
4
5
6

==> ok
```

Тот же пример с двумя различными переменными цикла дает совершенно предсказуемый результат:

```
(dolist (ite1 '(1 2 3) 'ok)
  (dolist (ite2 '(4 5 6) 'ok)
    (print ite1) (prints " ") (printline ite2)))

1 4
1 5
1 6
2 4
2 5
2 6
3 4
3 5
3 6

==> ok
```

DOTIMES

Эта функция обеспечивает выполнение блока кода заданное число раз. Синтаксис вызова функции следующий:

```
(dotimes (переменная_цикла
          число_повторений
          результат)

         (форма1)
         (форма2)
         …
         (формаn)
)
```

Вычисление происходит так. Переменная цикла последовательно получает значения от 0 до значения число_повторений-1. При каждом значении переменной цикла последовательно вычисляются формы (форма 1) - (форма n). По завершении цикла вычисляется форма-результат и возвращается в качестве результата DOTIMES. Вот как это выглядит:

Еще примеры:

```
(defun dtm (s)
       (dotimes (i 5 s)
       (print i)
       (setq s (+ s i))
))

0
1
2
3
4


(dotimes
   (i 5 t)
      (print 'i=)
      (printline i)
)

i=0
i=1
i=2
i=3
i=4

==> T
```

Достаточно интересно то, что переменную цикла в теле цикла можно менять! Никакого зацикливания при этом не произойдет:

```
(dotimes
   (i 5 t)
     (setq i -1)
     (print 'i=)
     (printline i)
)

i=-1
i=-1
i=-1
i=-1
i=-1

==> T
```

Для досрочного выхода из цикла можно использовать функцию RETURN:

```
(dotimes (i 10 t)

         (print 'i=)
         (printline i)
         (if (= i 7) (return 'ok) nil)
)

i=0
i=1
i=2
i=3
i=4
i=5
i=6
i=7

==> ok
```

В этом случае в качестве результата возвращается результат вызова RETURN.

Общий стандартный цикл (DO)

Хотя DOLIST и DOTIMES удобны и легки в использовании, они недостаточно гибки, чтобы использоваться для любых циклов. Например, что если вы захотите менять на каждом шаге несколько переменных параллельно? Или использовать произвольное выражение для проверки окончания цикла?

Если ни DOLIST, ни DOTIMES не подходят для ваших целей, есть наиболее общий циклу DO.

Там, где DOLIST и DOTIMES предоставляют только одну переменную цикла, DO позволяет держать любое число переменных и даёт полный контроль над тем, как они будут изменяться на каждом шаге цикла.

Так же можно определить проверку, которая говорит когда циклу завершиться и может предоставлять форму для вычисления в конце цикла, чтобы сформировать возвращаемое значение для всего DO в целом. Базовый шаблон выглядит так:

```
DO (
    (var1 init-form1 step-form1)
    (var2 init-form2 step-form2)
       . . .
    (varN init-formN step-formN)
)

      (stop-condition result-form1 . . . result-formM)
      (form1) (form2) . . . (formK)
)
```

init-form будет выполнена в начале цикла и полученное значение присвоено переменной var. Перед каждой последующей итерацией цикла, step-form будет выполнена и её значение присвоено var. Форма step-form необязательна; если её не будет, переменная останется с тем же значением от итерации к итерации, пока вы прямо не назначите ей новое значение в теле цикла. Если форма init-form не задана, переменной присваивается NIL.

В начале каждой итерации, после того, как все переменные цикла получили свои новые значения, выполняется форма stop-condition. До тех пор, пока она вычисляется в NIL, итерация происходит, выполняя form по порядку.

Когда вычисление формы stop-condition выдаст истину, будет вычислена форма result-form и значение, полученное в результате, будет возвращено как значение всего выражения DO.

Примеры

```
(do
((i 1 (+ i 1)))
((> i 10) (return 'ok))
(print i)
```

```
(DO (
      (i 1 (+ i 1))
```

```
      (j 1 (+ j 1)))
        ((> i 3) (print 'ok))
         (print (list i j))
)
```

```
(DO (
      (i 1 (+ i 1))
      (j 1 (+ j 1)))
      ((> j 5) (printline 'ok) (printline 'ok) (printline 'ok))
      (print 'i=) (print i) (prints " ") (print 'j=) (printline
j)
)

i=1 j=1
i=2 j=2
i=3 j=3
i=4 j=4
i=5 j=5
ok
ok
ok

==> ok
```

LOOP

Макрос LOOP на самом деле бывает двух видов: простой и расширенный. Простая версия проста как только можно: бесконечный цикл без связанных с ним переменных. Скелет выглядит так:

```
1(loop
2     body-form*)
```

Формы внутри тела выполняются каждый раз в процессе цикла, который будет длиться вечно, пока вы не используете RETURN, чтобы прервать его. Например, вы могли бы записать предыдущий DO цикл с простым LOOP.

Один из самых ценных и одновременно наименее документированных операторов в стандарте common lisp. Ценен он тем, что позволяет короче и яснее использовать функции высшего порядка (mapping) и рекурсивные операции. Также он позволяет использовать в Лиспе конструкции, более удобные для тех, кто привык работать с более традиционными языками.

Макрос Loop отличается от других операторов Лиспа, и больше напоминает конструкции Си или Паскаля. То есть, для чтения loop-выражений вам потребуется мыслить не только на языке Лиспа, но и на Паскале.

Loop-выражение может содержать выражения, определяющие итерируемые переменные; выражения, завершающие цикл (условно или безусловно); выражения, определяющие последовательность действий на каждой итерации; действия после завершения последней итерации. Кроме того, loop-выражения, как и полагается функциональной парадигме, возвращает какое-то значение. Редко когда приходится использовать все эти возможности, но оных позволяет их эффективно комбинировать.

Примеры

Итерация по списку, печатаем каждый элемент:

```
(loop for x in '(a b c d e)
    do (print x))

A
B
C
D
E
NIL
```

Итерация параллельно по двум спискам, собираем результат в ассоциированный список:

```
(loop for x in '(a b c d e)
    for y in '(1 2 3 4 5)
    collect (cons x y))

((A . 1) (B . 2) (C . 3) (D . 4) (E . 5))
```

Итерация с использованием счетчика. Возвращает список из элементов, полученных в результате каждой итерации.)

```
(loop for x from1 to 5
    for y = (* x 2)
    collect y)

(2 4 6 8 10)
```

Итерация с проходом по списку, параллельно с итерацией по счетчику. Окончание итерации по счетчику не определяется длиной списка. В теле loop-выражения используется условный оператор:

```
(loop for x in '(a b c d e)
    for y from 1
    when (> y 1)
```

```
    do (format t ", ")
    do (format t "~A" x))

A, B, C, D, E

NIL
```

То же самое, но через if:

```
(loop for x in '(a b c d e)
    for y from 1

    if (> y 1)
    do (format t ", ~A" x)
    else do (format t "~A" x))


A, B, C, D, E

NIL
```

Пример условно-досрочного завершения цикла:

```
(loop for x in '(a b c d e 1 2 3 4)
    until (numberp x)
    collect (list x 'foo))

((A FOO) (B FOO) (C FOO) (D FOO) (E FOO))
```

Также для организации условного завершения может использоваться "while":

```
(loop for x from 1
    for y = (* x 10)
    while (< y 100)
    do (print (* x 5))
    collect y)


5
10
15
20
25
30
35
40
45
(10 20 30 40 50 60 70 80 90)
```

Loop-выражения могут быть вложены друг в друга:

```
(loop for x from 1 to 10
    collect (loop for y from 1 to x
                collect y))

((1) (1 2) (1 2 3) (1 2 3 4) (1 2 3 4 5) (1 2 3 4 5 6) (1 2 3 4 5 6 7)
 (1 2 3 4 5 6 7 8) (1 2 3 4 5 6 7 8 9) (1 2 3 4 5 6 7 8 9 10))
```

На месте итерируемой переменной может стоять несколько переменных в виде списка. 3)

```
(loop for (a b) in '((x 1) (y 2) (z 3))
    collect (list b a))

((1 X) (2 Y) (3 Z))
```

Ключевое слово "return" (возврат) обеспечивает немедленный выход из цикла и (по желанию) возвращает значение, даваемое ему в виде первого аргумента:

```
(let ((s "alpha45"))
   (loop for i from 0 below (length s)
         for ch = (char s i)
         when (find ch "0123456789" :test #'eql)
         return ch))
```